

Python: exercises on functions

This notebook was generated in **student** mode.

Exercise 1: Greeting Function

Exercise Description

Create a function `greet` that takes a user's name as input and prints a greeting message. The function should have a default parameter that uses "Fabio" as the default name if no name is provided.

The function should print: "Hello, [name]! Welcome!"

Your solution

```
# Write your function here  
def greet(name="Fabio"):  
    pass
```

Test your solution

```
# Test the function with default parameter
import io
import sys
from contextlib import redirect_stdout

# Capture output from greet() with default parameter
f = io.StringIO()
with redirect_stdout(f):
    greet()
output = f.getvalue().strip()
assert output == "Hello, Fabio! Welcome!"
```

Test your solution

```
# Test the function with custom name
f = io.StringIO()
with redirect_stdout(f):
    greet("Alice")
output = f.getvalue().strip()
assert output == "Hello, Alice! Welcome!"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 2: Even or Odd Checker Function

Exercise Description

Write a function `is_even` that takes an integer as input and returns `True` if the number is even, `False` if it's odd.

Your solution

```
# Write your function here  
def is_even(number):  
    pass
```

Test your solution

```
# Test with even number  
assert is_even(4) == True  
assert is_even(0) == True  
assert is_even(100) == True
```

Test your solution

```
# Test with odd number
assert is_even(3) == False
assert is_even(1) == False
assert is_even(99) == False
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 3: Maximum of Two Numbers

Exercise Description

Create a function `max_of_two` that returns the larger of two numbers. The second parameter should have a default value of 0.

Your solution

```
# Write your function here
def max_of_two(a, b=0):
    pass
```

Test your solution

```
# Test with two positive numbers
assert max_of_two(5, 3) == 5
assert max_of_two(2, 8) == 8
assert max_of_two(10, 10) == 10
```

Test your solution

```
# Test with default parameter
assert max_of_two(5) == 5 # 5 > 0 (default)
assert max_of_two(-3) == 0 # -3 < 0 (default)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML; import urllib.parse as u; HTML('<a href="https://pyth
```

Exercise 4: Simple Calculator Function

Exercise Description

Implement a function `calculator` that takes two numbers and an operator (+, -, *, /) and returns the result of the operation. Handle division by zero and invalid operators appropriately:

- If the operator is invalid, return "Invalid operator".
- If division by zero is attempted, return "Division by zero error".

Your solution

```
# Write your function here  
def calculator(a, b, operator):  
    pass
```

Test your solution

```
# Test basic operations  
assert calculator(10, 5, '+') == 15  
assert calculator(10, 5, '-') == 5  
assert calculator(10, 5, '*') == 50  
assert calculator(10, 5, '/') == 2.0
```

Test your solution

```
# Test error cases  
assert calculator(10, 0, '/') == "Error: Division by zero"  
assert calculator(10, 5, '%') == "Invalid operator"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 5: Temperature Converter Function

Exercise Description

Write a function `celsius_to_fahrenheit` that converts a temperature from Celsius to Fahrenheit using the formula: $F = (C \times 9/5) + 32$

Your solution

```
# Write your function here
def celsius_to_fahrenheit(celsius):
    pass
```

Test your solution

```
# Test common temperature conversions
assert celsius_to_fahrenheit(0) == 32.0 # Freezing point
```

```
assert celsius_to_fahrenheit(100) == 212.0 # Boiling point
assert celsius_to_fahrenheit(25) == 77.0 # Room temperature
```

Test your solution

```
# Test negative temperatures
assert celsius_to_fahrenheit(-10) == 14.0
assert celsius_to_fahrenheit(-40) == -40.0 # Special case where C = F
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 6: Leap Year Checker Function

Exercise Description

Write a function `is_leap_year` that determines whether a given year is a leap year. A leap year is divisible by 4, but if it's divisible by 100, it must also be divisible by 400.

Your solution


```
# Write your function here
def is_leap_year(year):
    pass
```

Test your solution

```
# Test leap years
assert is_leap_year(2024) == True # Divisible by 4, not by 100
assert is_leap_year(2000) == True # Divisible by 400
assert is_leap_year(1600) == True # Divisible by 400
```

Test your solution

```
# Test non-leap years
assert is_leap_year(2023) == False # Not divisible by 4
assert is_leap_year(1900) == False # Divisible by 100 but not by 400
assert is_leap_year(2100) == False # Divisible by 100 but not by 400
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 7: Circle Area Calculator

Exercise Description

Implement a function `circle_area` that calculates the area of a circle given its radius. Use the formula: $\text{Area} = \pi \times \text{radius}^2$

Your solution

```
import math

# Write your function here
def circle_area(radius):
    pass
```

Test your solution

```
# Test with common radius values
import math
assert abs(circle_area(1) - math.pi) < 0.0001 # Area of unit circle
assert abs(circle_area(2) - 4 * math.pi) < 0.0001 # Area = 4π
```

Test your solution

```
# Test with larger radius
assert abs(circle_area(5) - 25 * math.pi) < 0.0001 # Area = 25π
assert circle_area(0) == 0 # Zero radius
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 8: Vowel Checker Function

Exercise Description

Create a function `is_vowel` that checks whether a given character is a vowel (a, e, i, o, u) in either uppercase or lowercase.

Your solution

```
# Write your function here
def is_vowel(char):
    pass
```

Test your solution

```
# Test with lowercase vowels
assert is_vowel('a') == True
```

```
assert is_vowel('e') == True
assert is_vowel('i') == True
assert is_vowel('o') == True
assert is_vowel('u') == True
```

Test your solution

```
# Test with uppercase vowels and consonants
assert is_vowel('A') == True
assert is_vowel('E') == True
assert is_vowel('b') == False
assert is_vowel('Z') == False
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 9: Simple Interest Calculator

Exercise Description

Write a function `simple_interest` that calculates simple interest given principal amount, rate of interest, and time period. Use the formula: Simple Interest = (Principal × Rate × Time) / 100

Your solution

```
# Write your function here
def simple_interest(principal, rate, time):
    pass
```

Test your solution

```
# Test with common values
assert simple_interest(1000, 5, 2) == 100.0 # $1000 at 5% for 2 years
assert simple_interest(5000, 10, 1) == 500.0 # $5000 at 10% for 1 year
```

Test your solution

```
# Test with different scenarios
assert simple_interest(2000, 7.5, 3) == 450.0 # $2000 at 7.5% for 3 years
assert simple_interest(1000, 0, 5) == 0.0 # 0% interest rate
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 10: Absolute Value Function

Exercise Description

Create a function `absolute` that returns the absolute value of a number (the non-negative value without regard to its sign).

Your solution

```
# Write your function here
def absolute(num):
    pass
```

Test your solution

```
# Test with positive numbers
assert absolute(5) == 5
assert absolute(10.5) == 10.5
assert absolute(0) == 0
```

Test your solution

```
# Test with negative numbers
assert absolute(-5) == 5
assert absolute(-10.5) == 10.5
assert absolute(-100) == 100
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 11: Maximum of Three Numbers

Exercise Description

Write a function `max_of_three` that returns the largest of three numbers.

Your solution

```
# Write your function here
def max_of_three(a, b, c):
    pass
```

Test your solution

```
# Test with different maximum positions
assert max_of_three(10, 5, 8) == 10 # First is maximum
assert max_of_three(3, 15, 7) == 15 # Second is maximum
assert max_of_three(4, 6, 20) == 20 # Third is maximum
```

Test your solution

```
# Test with equal values
assert max_of_three(5, 5, 5) == 5 # All equal
assert max_of_three(10, 10, 8) == 10 # Two equal maximums
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 12: Power Function

Exercise Description

Write a function `power` that calculates the power of a number (base raised to the exponent).

Your solution

```
# Write your function here  
def power(base, exponent):  
    pass
```

Test your solution

```
# Test with common power calculations  
assert power(2, 3) == 8 # 2^3 = 8  
assert power(5, 2) == 25 # 5^2 = 25  
assert power(10, 0) == 1 # Any number to power 0 is 1
```

Test your solution

```
# Test with edge cases  
assert power(3, 1) == 3 # Any number to power 1 is itself  
assert power(2, 4) == 16 # 2^4 = 16
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 13: Calculate Body Mass Index (BMI)

Exercise Description

Write a function `calculate_bmi(weight, height)` that returns the Body Mass Index given weight (in kilograms) and height (in meters). The BMI is computed as $\text{weight} / (\text{height}^2)$.

Your solution

```
# Write your function here
def calculate_bmi(weight, height):
    pass
```

Test your solution

```
# typical values
assert round(calculate_bmi(70, 1.75), 2) == 22.86
assert round(calculate_bmi(60, 1.6), 2) == 23.44
```

Test your solution

```
# edge-like scenarios
assert round(calculate_bmi(90, 2.0), 2) == 22.5
assert round(calculate_bmi(45, 1.5), 2) == 20.0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 14: Minimum of Two Numbers

Exercise Description

Create a function `min_of_two(a, b)` that returns the smaller of two numbers `a` and `b`.

Your solution

```
# Write your function here
def min_of_two(a, b):
    pass
```

Test your solution

```
# typical comparisons
assert min_of_two(3, 5) == 3
assert min_of_two(-1, 2) == -1
assert min_of_two(7, 7) == 7
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 15: Determine Grade Based on Score

Exercise Description

Write a function `determine_grade(score)` that returns a letter grade based on a numeric score from 0 to 100 inclusive. Use the following scale: A for 90+, B for 80–89, C for 70–79, D for 60–69, F for below 60.

Your solution

```
# Write your function here
def determine_grade(score):
```

pass

Test your solution

```
# boundary checks
assert determine_grade(100) == 'A'
assert determine_grade(90) == 'A'
assert determine_grade(89) == 'B'
assert determine_grade(80) == 'B'
assert determine_grade(79) == 'C'
assert determine_grade(70) == 'C'
assert determine_grade(69) == 'D'
assert determine_grade(60) == 'D'
assert determine_grade(59) == 'F'
assert determine_grade(0) == 'F'
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 16: Convert Fahrenheit to Celsius

Exercise Description

Implement a function `fahrenheit_to_celsius(fahrenheit)` that converts a temperature in Fahrenheit to Celsius using the formula $C = (F - 32) * 5 / 9$. Return the Celsius value as a float.

Your solution

```
# Write your function here
def fahrenheit_to_celsius(fahrenheit):
    pass
```

Test your solution

```
# known reference points
assert round(fahrenheit_to_celsius(32), 2) == 0.00
assert round(fahrenheit_to_celsius(212), 2) == 100.00
assert round(fahrenheit_to_celsius(68), 2) == 20.00
```

Test your solution

```
# additional checks
assert round(fahrenheit_to_celsius(14), 2) == -10.00
assert round(fahrenheit_to_celsius(41), 2) == 5.00
```

Debug your solution

